

Outage Costs Calculation Documentation

1 Introduction

The `outage_costs.py` script calculates expected outage costs using a simplified probabilistic reliability approach. It models outage rates by terrain and construction type, applies a multiplicative duration model, and uses piecewise value of lost load (VoLL) to calculate costs.

2 Method Overview

2.1 Purpose

This script calculates outage costs by:

1. Calculating segment-specific outage rates by terrain and construction type
2. Applying duration multipliers based on construction type
3. Calculating unserved energy per event
4. Applying piecewise VoLL to calculate cost per event
5. Computing Expected Annual Cost (EAC)
6. Calculating nominal total cost using growing annuity
7. Computing present value using growing annuity formula

2.2 Calculation Approach

The outage cost calculation follows this structure:

$$\lambda_{segment} = \text{Miles}_{terrain} \times \text{Outage Rate}_{terrain, construction} \quad (1)$$

$$H_{eff} = H_{base} \times \text{Duration Multiplier}_{construction} \quad (2)$$

$$\text{MW Lost} = \phi \times \text{Capacity} \quad (3)$$

$$\text{Unserved MWh} = H_{eff} \times \text{MW Lost} \quad (4)$$

$$\text{Cost/Event} = \text{Piecewise VoLL}(\text{Unserved MWh}) \quad (5)$$

$$\text{EAC} = \sum_{\text{terrains}} \lambda_{segment} \times \text{Cost/Event} \quad (6)$$

$$\text{Nominal Total} = \text{EAC} \times \frac{(1+g)^N - 1}{g} \quad (7)$$

$$\text{PV} = \text{EAC} \times \frac{1 - \left(\frac{1+g}{1+d}\right)^N}{d - g} \quad (8)$$

where:

- λ = annual outage rate (outages/year)
- H_{eff} = effective outage duration (hours)
- ϕ = capacity at risk factor
- g = risk growth rate
- d = discount rate
- N = project lifetime

2.3 Inputs and Outputs

Inputs:

- Project technical details (construction type, capacity, project lifetime, delay years, construction years)
- Terrain miles (`load_terrain_miles`)
- Outage parameters (outage rates, duration by terrain, duration multipliers, VoLL tiers, capacity at risk factor, growth rate)
- Discount rate (`discount_rate_type` from financing)
- Financing data (for discount rate)

Outputs:

- Total annual outage rate (lambda)

- Expected Annual Cost (EAC)
- Outage details by terrain segment
- Nominal total cost (growing annuity)
- Present value cost (growing annuity with discounting)

2.4 Integration with Other Scripts

This script integrates with:

- `smart_loaders.py` — `load_project_technical_details`, `load_financing_details`, `load_outage_costs`, `load_terrain_miles`, `get_financing_data_raw`
- `constants.py` — `MIN_DISCOUNT_RATE`, `GROWTH_RATE_TOLERANCE`
- `financial_utils.py` — `get_discount_rate_from_config`, `calculate_growing_annuity_pv`, `validate_discount_rate`, `calculate_nominal_growing_series`
- `calculation_utils.py` — `build_category_string`, `normalize_construction_type_for_yaml`
- `smart_output.py` — `CTCCOutputManager` (`add_outage_costs`, `write_batch_summary`)

3 Key Functions

3.1 `get_discount_rate_from_config` (from `financial_utils`)

The script uses `financial_utils.get_discount_rate_from_config(config_yaml, financing_yaml, rate_key="discount_rate_type")`. In `main()`, it is called with `outage_yaml`, `get_financing_data_raw()`, and `rate_key="discount_rate_type"`. Returns (`discount_rate`, `source.description`).

3.2 `calculate_voll_cost_piecewise`(`duration_hours`, `mw_lost`, `tiers`)

Calculates total cost using piecewise VoLL with tiered pricing.

Parameters:

- `duration_hours` (float): Duration of outage in hours
- `mw_lost` (float): MW capacity lost during outage
- `tiers` (list): List of tier dicts with `max_hours` and `value_per_mwh`

Returns:

- `total_cost` (float): Total cost for the outage

Key Logic:

```

1 # Piecewise VoLL: Hours 0-4 at tier 1, hours 4-24 at tier 2, hours
   24+ at tier 3
2 total_cost = 0
3 hours_remaining = duration_hours
4 prev_threshold = 0
5
6 for tier in tiers:
7     max_hours = float("inf") if tier["max_hours"] == ".inf" else
       tier["max_hours"]
8     value_per_mwh = tier["value_per_mwh"]
9     hours_in_tier = min(hours_remaining, max_hours - prev_threshold)
10    if hours_in_tier > 0:
11        mwh_in_tier = hours_in_tier * mw_lost
12        total_cost += mwh_in_tier * value_per_mwh
13        hours_remaining -= hours_in_tier
14        prev_threshold = max_hours
15    if hours_remaining <= 0:
16        break

```

3.3 calculate_outage_costs(outage_yaml, construction_type, terrain_miles, capacity_mw, project_lifetime, discount_rate)

Calculates expected outage costs using simplified outage rate model.

Parameters:

- outage_yaml (dict): Loaded outage YAML data
- construction_type (str): Construction type (overhead, underground, subsea)
- terrain_miles (dict): Dictionary of terrain type to miles
- capacity_mw (float): Project capacity in MW
- project_lifetime (int): Project lifetime in years
- discount_rate (float): Discount rate for PV calculation
- delay_years (float, optional): Years of delay before construction starts
- construction_years (float, optional): Years of construction (PV annuity starts after delay + construction)

Returns:

- Dictionary containing:
 - lambda_total (float): Total annual outage rate
 - EAC (float): Expected Annual Cost

- outage_by_terrain (dict): Outage details by terrain segment
- capacity_at_risk (float): Capacity at risk factor
- growth_rate (float): Risk growth rate
- nominal_total (float): Total nominal cost
- pv_cost (float): Present value cost

Key Logic:

```

1 yaml_construction_type = normalize_construction_type_for_yaml(
    construction_type)
2
3 for terrain, miles in terrain_miles.items():
4     if miles > 0:
5         outage_rate = outage_rates[yaml_construction_type].get(
            terrain, 0.0)
6         lambda_segment = miles * outage_rate
7
8         H_base = duration_by_terrain.get(terrain, 0)
9         duration_mult = duration_multiplier.get(
            yaml_construction_type, 1.0)
10        H_eff = H_base * duration_mult
11
12        mw_lost_per_event = capacity_at_risk * capacity_mw
13        unserved_mwh_per_event = H_eff * mw_lost_per_event
14        cost_per_event = calculate_voll_cost_pieewise(
15            H_eff, mw_lost_per_event, voll_tiers
16        )
17        annual_cost = lambda_segment * cost_per_event
18        # ... store in outage_by_terrain, accumulate lambda_total,
19        # EAC
20
21 nominal_total = calculate_nominal_growing_series(...)
22 pv_cost = calculate_growing_annuity_pv(..., delay_years=...,
    construction_years=...)

```

3.4 main()

Main execution function that orchestrates the outage cost calculation and output.

4 Example

The following example demonstrates a typical usage of the outage cost calculation:

```

1 # Example: 500 MW Overhead AC line
2 # Category: "Overhead/AC/500MW/Advanced Aluminum/NA"
3 #

```

```

4 # Inputs (from YAML):
5 #   - Terrain distribution:
6 #     * Forested: 40 miles, outage rate: 0.05 outages/mi/yr
7 #     * Farmland: 60 miles, outage rate: 0.02 outages/mi/yr
8 #   - Duration: 8h (forested), 4h (farmland)
9 #   - Duration multiplier: 1.0 (overhead)
10 #   - Capacity at risk: 50%
11 #   - VoLL tiers:
12 #     * 0-4h: $10,000/MWh
13 #     * 4-24h: $5,000/MWh
14 #     * 24h+: $2,000/MWh
15 #   - Risk growth rate: 1% per year
16 #   - Project lifetime: 50 years
17 #   - Discount rate: 3% (social)
18 #
19 # Calculation:
20 #   Forested segment:
21 #     Outages/year = 40 * 0.05 = 2.0
22 #     Duration = 8h * 1.0 = 8h
23 #     MW lost = 0.5 * 500 = 250 MW
24 #     Unserved MWh = 8 * 250 = 2000 MWh
25 #     Cost/event = (4h * 250MW * $10k) + (4h * 250MW * $5k) = $15M
26 #     Annual cost = 2.0 * $15M = $30M/year
27 #
28 #   Farmland segment:
29 #     Outages/year = 60 * 0.02 = 1.2
30 #     Duration = 4h * 1.0 = 4h
31 #     MW lost = 250 MW
32 #     Unserved MWh = 4 * 250 = 1000 MWh
33 #     Cost/event = 4h * 250MW * $10k = $10M
34 #     Annual cost = 1.2 * $10M = $12M/year
35 #
36 #   Total EAC = $42M/year
37 #   Nominal total = $42M * ((1.01^50 - 1) / 0.01) = $2.7B
38 #   PV = $42M * ((1 - (1.01/1.03)^50) / (0.03 - 0.01)) = $1.2B
39 #
40 # Output:
41 #   - Total outages/year: 3.2 events/year
42 #   - EAC: $42M/year
43 #   - Nominal total: $2.7B
44 #   - Present value: $1.2B

```

Integration Example:

```

1 # When called from ctcc.py:
2 csv_manager = CTCCOutputManager()
3 # Results dict already has all needed values
4 csv_manager.add_outage_costs(results)

```

```
5 csv_manager.write_batch_summary()
```

5 Key Implementation Details

5.1 Piecewise Value of Lost Load (VoLL)

The script uses a tiered VoLL structure where different outage durations have different values:

- **Tier 1** (0-4 hours): Highest value (e.g., \$10,000/MWh) - immediate impacts
- **Tier 2** (4-24 hours): Medium value (e.g., \$5,000/MWh) - short-term impacts
- **Tier 3** (24+ hours): Lower value (e.g., \$2,000/MWh) - longer-term impacts

This reflects the reality that shorter outages have higher economic impact per MWh.

5.2 Multiplicative Duration Model

The effective duration is calculated as:

$$H_{eff} = H_{base} \times \text{Duration Multiplier}_{construction}$$

Different construction types have different duration multipliers. For example, underground lines may have shorter repair times (lower multiplier) than overhead lines.

5.3 Capacity at Risk Factor

The capacity at risk factor (ϕ) represents the fraction of project capacity that is lost during an outage. This accounts for partial outages and system redundancy.

5.4 Growing Annuity Formula

Similar to wildfire costs, the script accounts for increasing outage risk over time using a growing annuity formula. Present value uses `calculate_growing_annuity_pv` with `delay_years` and `construction_years` so the annuity stream starts after construction (delay + construction years).

5.5 AFUDC Treatment

Outage costs are probabilistic future losses, not capital costs. Therefore, they are not AFUDC-eligible.